Twinkle — building the installer

How to turn the source code into something you can hand to a clinic - either one self-contained setup file, or a ClickOnce deployment that updates itself.

Handing it over and installing it are a separate job, written up in INSTALLING.md. This document stops when the file exists.

What you need, once

Windows	10 or 11, 64-bit. IExpress, which does the packing, ships with it
.NET SDK 10	`dotnet --version` should print `10.` or higher
The repository	Cloned, and it builds — `dotnet build HMS_WPF.slnx`

Nothing else. No Inno Setup, no WiX, no Visual Studio. The whole thing is one PowerShell script and a tool Windows already has.

> Visual Studio *is* needed for the ClickOnce route at the end of this document, which is the one exception.

The four steps

1. Set the version

Open `src/Pharma.App/Pharma.App.csproj` and change one line:

```
<Version>1.0.1</Version>
```

That single field becomes:

- the version in the navigation sidebar, which support asks people to read out,
- the version listed under **Add or remove programs**,
- the assembly and file version stamped on the executable.

> **Do this before every release.** Two different builds both calling themselves 1.0.0 is the fastest way to spend an afternoon on a bug that was fixed weeks ago. There is nothing in the build that will stop you.

2. Run the tests

```
dotnet test tests\Pharma.Tests\Pharma.Tests.csproj
```

The 240-odd unit tests take about half a minute and cover the pricing, the GST arithmetic, pack sizes and the data health checks — the parts where a mistake costs somebody money.

The UI tests are worth running before a release you are actually shipping:

```
dotnet test tests\Pharma.UiTests\Pharma.UiTests.csproj
```

They drive the real application through the real screens and take six or seven minutes. They open and close windows while they run, so leave the machine alone.

3. Build the installer

```
powershell -ExecutionPolicy Bypass -File scripts\make-installer.ps1
```

Two or three minutes, most of it compressing. It prints its progress:

```
1/3  Building the application as a single file ...
     72.9 MB
2/3  Writing the package definition ...
3/3  Packing (a 73 MB payload takes a minute or two) ...

Setup file:  C:\HMS\Setup\TwinkleHMSSetup.exe  (66.6 MB)
```

Somewhere other than `C:\HMS\Setup`:

```
powershell -ExecutionPolicy Bypass -File scripts\make-installer.ps1 -OutputDir D:\Handover
```

4. Check what came out

Two quick checks, worth the minute they cost.

The file is about the right size. Roughly 65–70 MB. If it is a few hundred kilobytes, the payload did not go in and you have packaged an installer that installs nothing.

It contains what it should. Unpack it without running it:

```
"C:\HMS\Setup\TwinkleHMSSetup.exe" /C /Q /T:%TEMP%\check
```

`%TEMP%\check` should hold exactly three files — `TwinkleHMS.exe` at about 73 MB, `install.cmd` and `install.ps1`.

> **Then test it on a machine that is not this one.** A clean PC or a virtual machine with no .NET installed and no `C:\HMS` folder. Installing on the machine that built it proves almost nothing: everything it needs is already there. This is the step that catches a missing runtime, and it is the step everybody skips.

What the script actually does

Worth knowing for the day it breaks.

1. **Publishes the application** self-contained, single-file and compressed. Self-contained means the .NET runtime is inside the executable, so the clinic PC needs nothing installed first. That is what makes it 73 MB rather than 5.
2. **Throws away everything else** the publish produced — debug symbols, the Lato fonts QuestPDF ships as package content, `appsettings.json`. None of it is needed beside a single-file build.
3. **Writes an IExpress definition file**, an ini-style `.sed`. IExpress takes no arguments; the file is how you talk to it.
4. **Runs IExpress**, which packs the executable and the two install scripts into one self-extracting file and sets it to run `install.cmd` afterwards.

`scripts/installer/install.ps1` is the part that runs on the clinic PC: it asks for administrator rights once, refuses if Twinkle is open, copies the executable to `C:\HMS\App`, makes the data folders, creates the shortcuts and registers the uninstaller.

When the build fails

What you see	What it is
`The string is missing the terminator`	A non-ASCII character got into a `.ps1`. Windows PowerShell reads these as ANSI without a byte order mark, and one em dash in a string stops the file parsing. Keep the scripts plain ASCII
`Publish failed`	Build the solution on its own to see the real compiler error — the publish output hides it
`TwinkleHMS.exe was not produced`	The publish succeeded but wrote somewhere else. Check `-o` against the staging folder in the script
`IExpress did not produce ...`	Usually `TargetName` in the `.sed` pointing at a folder that does not exist. The script creates it, so this means the path is wrong
The file is a few hundred KB	The payload was not picked up. Check that the staging folder still held the executable when IExpress ran
`being used by another process`	Twinkle is open, or a previous run is still holding the file. Close it

---

Signing, when it is worth it

The setup file is not code-signed. On the clinic PC that means:

- "Windows protected your PC" on first run — **More info** → **Run anyway**,
- the publisher shown as unknown,
- antivirus occasionally quarantining it, since an unsigned self-extracting executable is a common false positive.

For one or two clinics, warning people that the message is coming is enough. For wider distribution, buy a code-signing certificate and sign both `TwinkleHMS.exe` and `TwinkleHMSSetup.exe` with `signtool`. The warnings stop.

Building it with ClickOnce instead

The other supported route. Worth it for one reason: **it can update itself.**

Which to use

	Setup file	ClickOnce
Files to send	**One**	Three, which must stay together
Needs .NET on the clinic PC	No, it is inside	**Yes** — the bootstrapper fetches it, which needs internet that first time
Needs an administrator	Yes, once	**No** — it installs per user
Desktop shortcut	Yes	**Start menu only**
Automatic updates	No	**Yes**, from a shared folder
Needs Visual Studio to build	No	**Yes**

One PC and a pen drive: use the setup file. Several PCs on a clinic network, or you expect to ship fixes often: ClickOnce earns its extra pieces.

You need Visual Studio

Not just the SDK. ClickOnce builds `setup.exe` with a task called `GenerateBootstrapper`, which only exists in the .NET Framework MSBuild that ships with Visual Studio:

```
error MSB4803: The task "GenerateBootstrapper" is not supported on the
.NET Core version of MSBuild.
```

So `dotnet msbuild` and `dotnet publish` cannot do it, whatever arguments you give them.

Build it

```
powershell -ExecutionPolicy Bypass -File scripts\publish-clickonce.ps1
```

The script finds Visual Studio's MSBuild for you, publishes, and zips the result so there is one file to send:

```
Published to src\Pharma.App\bin\publish
     Application Files
     setup.exe
     TwinkleHMS.application

2/2  Zipping ...
     version 1.0.0.1
     1 older version(s) in the publish folder left out of the zip

One file to send:  C:\HMS\Setup\TwinkleHMS-ClickOnce.zip  (69.5 MB)
```

From Visual Studio instead: right-click **Pharma.App** → **Publish** → **ClickOnceProfile** → **Publish**. Same output, same folder.

`-NoZip` leaves the three pieces loose, which is what you want when publishing to a shared folder rather than carrying it.

> `Application Files` **keeps every revision you have ever published.** Each one is a full copy of the application. Two publishes and the folder holds 330 MB. That is correct for a shared folder — a PC part-way through an update still needs the version it is coming from — but it is dead weight in a zip somebody carries once, so the script packs only the version the manifest points at. Delete the folder occasionally if you publish a lot.

Installing it at the clinic

1. Copy the zip over and **unzip it**.
2. Run `setup.exe` **from the unzipped folder**. Running it from inside the zip viewer fails — the files it needs are still zipped, and the error does not say so.
3. If .NET 10 Desktop Runtime is missing, the bootstrapper offers to download it. **That needs internet on the clinic PC.** On a machine with none, install the runtime by hand first, or use the setup file instead.
4. No administrator prompt. It installs per user, under `%LocalAppData%\Apps\2.0\`.
5. The shortcut is in the **Start menu** only, under Twinkle Children's Hospital. There is no desktop icon — ClickOnce does not make one.

`C:\HMS\DB` is untouched, exactly as with the setup file. The database never lives beside the program, which is what makes either route safe to re-run.

Turning on automatic updates

This is the reason to choose ClickOnce, and it is off in the profile today.

In `src/Pharma.App/Properties/PublishProfiles/ClickOnceProfile.pubxml`:

```
<UpdateEnabled>True</UpdateEnabled>
<UpdateMode>Foreground</UpdateMode>
<InstallFrom>Unc</InstallFrom>
<PublishUrl>\\clinic-server\twinkle\</PublishUrl>
```

Publish to that share once. Every PC installed from it then checks the share at launch and updates itself. Publish a fix in the morning and the counter has it by the afternoon, with nobody carrying anything.

Leave the old versions in `Application Files` when you do this — that is what they are for.

Two things to know before you rely on it

- **`appsettings.json` is replaced on every update.** It sits in the versioned folder, and an update installs a new one. A clinic that edited its paths would lose them. The defaults are `C:\HMS\DB` and `C:\HMS\DBBackup`, so this only bites if someone has changed them.
- **The manifests are not signed.** `SignManifests` is `False`, so the same unknown-publisher warnings apply as to the setup file.

Publishing to this machine

`scripts/publish.ps1` installs straight into `C:\HMS\App` on the machine you run it on and makes a desktop shortcut. For the development PC — not for handover.

Rebuilding this document

```
node scripts\docs-to-pdf\guide_to_pdf.mjs docs\BUILDING_THE_INSTALLER.md docs\BUILDING_THE_INSTALLER.pdf
```